Technical Metamodel Specification

HEXAGEN-SMM: Hexagonal Architecture Spring Boot Metamodel

Grupo EGL

Alexis Ciclos

Juan Ante

Juan Camilo Rodallega

Juan Camilo Zarta

Universidad del Cauca

Facultad de Ingeniería Electrónica y Telecomunicaciones (FIET)

IDIS Research Group

Software Engineering Line

Project	Tópicos Avanzados de Ingeniería de Software
Version	1.0
Date	May 22, 2026
Document ID	TR TAIS-SE-HEXAGEN-V1.0
Type	Technical
Status	Final
Availability	Internal

© May 22, 2026, University of Cauca — IDIS Research Group

Revision History

Date	Version	Description	Author
22/05/2026	1.0	First complete draft — Hexagonal Architecture Metamodel specification	Grupo EGL

Document Information:

- **Author:** Grupo EGL
- **Date:** May 22, 2026
- **Document ID:** TR TAIS-SE-HEXAGEN-V1.0
- **Type:** Technical
- **Status:** Final
- **Availability:** Internal
- **Copyright:** ©2026 Universidad del Cauca — IDIS Research Group

Abstract

English. This document specifies the HEXAGEN-SMM metamodel, a domain-specific modeling language designed to represent Spring Boot applications structured under the Hexagonal Architecture (Ports and Adapters) pattern. The metamodel abstracts the structural concerns of a multi-layer Java application — Domain, Application, and Infrastructure — into a set of well-defined metaclasses, relationships, and constraints. It is intended for use in model-driven engineering tasks including model validation (EVL) and code generation (EGL). The metamodel is expressed using the Eclipse Modeling Framework (EMF) and the Emfatic notation.

Español. Este documento especifica el metamodelo HEXAGEN-SMM, un lenguaje de modelado específico de dominio diseñado para representar aplicaciones Spring Boot estructuradas bajo el patrón de Arquitectura Hexagonal (Puertos y Adaptadores). El metamodelo abstrae los aspectos estructurales de una aplicación Java multicapa — Dominio, Aplicación e Infraestructura — en un conjunto de metaclases, relaciones y restricciones bien definidas. Está destinado a su uso en tareas de ingeniería dirigida por modelos, incluyendo validación de modelos (EVL) y generación de código (EGL).

Contents

1 Scope

This specification defines the HEXAGEN-SMM metamodel for representing Spring Boot applications that follow the Hexagonal Architecture (also known as Ports and Adapters) pattern. The metamodel includes:

- A **type system** based on the Composite pattern, supporting primitive types, custom types (cross-references to modeled classes), collection types, and map types.
- A **three-layer hexagonal structure** composed of a Domain Layer, an Application Layer, and an Infrastructure Layer, all scoped within named Bounded Contexts.
- **Domain components**: Entities with explicit identity types, Value Objects, and Repository Ports (interfaces).
- **Application components**: Services that orchestrate domain logic through ports.
- **Infrastructure components**: REST Controllers (adapters-in), Repository Adapters (adapters-out), and Data Transfer Objects (DTOs).
- **Class members**: typed Attributes, Methods with return types and parameters, Endpoints with HTTP semantics, and Parameters with binding kind (`PATH`, `QUERY`, `BODY`).

The metamodel is intended as an intermediate representation for model management tasks — specifically model validation (EVL) and code generation (EGL) targeting Spring Boot projects. It does not cover persistence annotations (JPA), security, asynchronous messaging, or multi-module project structure, which may be addressed in future extensions.

2 Background: Hexagonal Architecture and Spring Boot

2.1 Basic Concepts

Hexagonal Architecture, introduced by Alistair Cockburn [1], is a software design pattern that separates the core business logic of an application from its external concerns (databases, UIs, external services) through the use of **Ports** and **Adapters**. The fundamental goal is to keep the domain model pure and independent of frameworks, allowing it to be tested and evolved in isolation.

The pattern organizes an application into three concentric layers:

- **Domain Layer:** Contains business entities, value objects, and repository port interfaces. This layer has zero dependency on any framework.
- **Application Layer:** Contains service classes (use cases) that orchestrate domain objects and depend on repository ports.
- **Infrastructure Layer:** Contains adapters implementing the ports — REST controllers (inbound adapters), repository implementations (outbound adapters), and DTOs.

Spring Boot [2] is a Java framework that enables rapid development of production-ready applications. It provides auto-configuration, an embedded server, and convention-over-configuration defaults, making it the de facto standard for implementing hexagonal architectures in Java.

A **Bounded Context** [3] is a DDD concept that defines the boundary within which a particular domain model applies. In HEXAGEN-SMM, each Bounded Context encapsulates

its own Domain, Application, and Infrastructure layers.

The following concepts are central to this metamodel:

- **Entity:** A domain object with a persistent identity (e.g., a `Book` identified by its UUID).
- **Value Object:** A domain object defined solely by its attributes, with no identity.
- **Repository Port:** A Java interface in the domain layer that declares data access contracts without exposing implementation details.
- **Service:** A use-case class that depends on one or more Repository Ports and implements business logic.
- **Controller:** An inbound adapter that exposes HTTP endpoints and delegates to a Service.
- **Repository Adapter:** An outbound adapter implementing a Repository Port using a concrete mechanism (file, database, external API).
- **DTO (Data Transfer Object):** A lightweight object used to transfer data across layer boundaries.

2.2 Example: Library Management System

A classic example is a Library Management System. It has a single Bounded Context containing:

- A **Domain Layer** with a `Book` entity (UUID identity), a `Loan` entity (referencing `Book`), and a `BookRepositoryPort` interface.
- An **Application Layer** with a `LibraryService` that depends on `BookRepositoryPort`.
- An **Infrastructure Layer** with a `BookController` (REST adapter at `/api/books`), a `JsonBookRepositoryAdapter` (implements `BookRepositoryPort` via JSON), and a `BookDto`.

This instance model is used throughout the document to illustrate metamodel concepts and to serve as the test model for validation (EVL) and code generation (EGL).

3 Conformance

An implementation is conformant with this specification if it supports the creation, storage, and exchange of models that adhere to the metaclasses, attributes, and relationships defined in Section 8, and if it respects the constraints described in Section 10.

Specifically, a conformant model instance:

- MUST define at least one `BoundedContext` within its root `Application`.
- MUST ensure every `Controller` references a `Service` defined within the same `ApplicationLayer`.
- MUST ensure every `RepositoryAdapter` implements a `RepositoryPort` defined within the same `DomainLayer`.
- MUST conform to the type system defined in Section 8.2 for all typed features.

4 Normative References

- **OMG MOF 2.5.1** — Meta-Object Facility Core Specification.

- **Eclipse EMF** — Eclipse Modeling Framework (implementation reference).
- **RFC 2119** — Key words for use in RFCs to Indicate Requirement Levels.
- **Epsilon EVL** — Epsilon Validation Language [4].
- **Epsilon EGL** — Epsilon Generation Language [4].
- **Emfatic** — Textual syntax for EMF metamodels.

5 Terms and Definitions

Term	Definition
Application	The root model element representing the entire Spring Boot application, including its metadata and bounded contexts.
Bounded Context	A named boundary that encapsulates a cohesive domain model along with its application and infrastructure concerns.
Domain Layer	The innermost layer containing entities, value objects, and port interfaces, free of framework dependencies.
Application Layer	The layer containing service classes (use cases) that orchestrate domain logic via repository ports.
Infrastructure Layer	The outermost layer containing inbound adapters (controllers), outbound adapters (repository adapters), and DTOs.
Entity	A domain object with a persistent identity.
Value Object	A domain object defined entirely by its attributes, with no independent identity.
Repository Port	A Java interface in the domain layer that declares data access operations.
Repository Adapter	A concrete class in the infrastructure layer that implements a Repository Port.
Service	An application-layer class implementing one or more use cases.
Controller	An infrastructure-layer class that receives HTTP requests and delegates to a Service.
Endpoint	A specific HTTP operation exposed by a Controller, with an HTTP method and URL path.
DTO	Data Transfer Object — a plain object carrying data between layers.
Type	The abstract base of the type system; every typed feature references a subtype of Type.

6 Symbols (Notation)

In the diagrams that accompany this specification, the following notation is used:

- **Boxes** represent metaclasses.
- **Lines with solid diamonds** represent composition (containment) — the child cannot exist without the parent.

- **Lines with open arrowheads** represent non-containment references (cross-references).
- **Lines with open triangle arrowheads** represent inheritance (generalization).
- **Attributes** are shown inside the box below the class name.
- *Abstract metaclasses* are shown in italics.
- **Enumerations** are shown with a red `E` icon in Sirius/Obeo diagrams.

7 Additional Information

7.1 Conventions

The keywords **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHOULD**, **RECOMMENDED**, and **OPTIONAL** are interpreted as described in RFC 2119.

In metamodel descriptions:

- **Containment** references are marked as `val` in Emfatic notation.
- **Non-containment** references are marked as `ref` in Emfatic notation.
- Multiplicity `[*]` means zero or more; `[1]` means exactly one.

7.2 How to Read this Specification

- Section 2 provides background on Hexagonal Architecture and Spring Boot.
- Section 8 presents the core metaclasses organized thematically.
- Section 9 shows a complete model instance (Library Management System).
- Section 10 lists the Well-Formedness Constraints.

8 Core Classes

This section specifies all metaclasses of the HEXAGEN-SMM metamodel. The metamodel is defined using EMF and follows the Emfatic syntax. Figure 1 provides a general overview; Figure 2 shows the containment hierarchy; Figure 4 shows the detailed class structure; Figure 3 shows the type system.

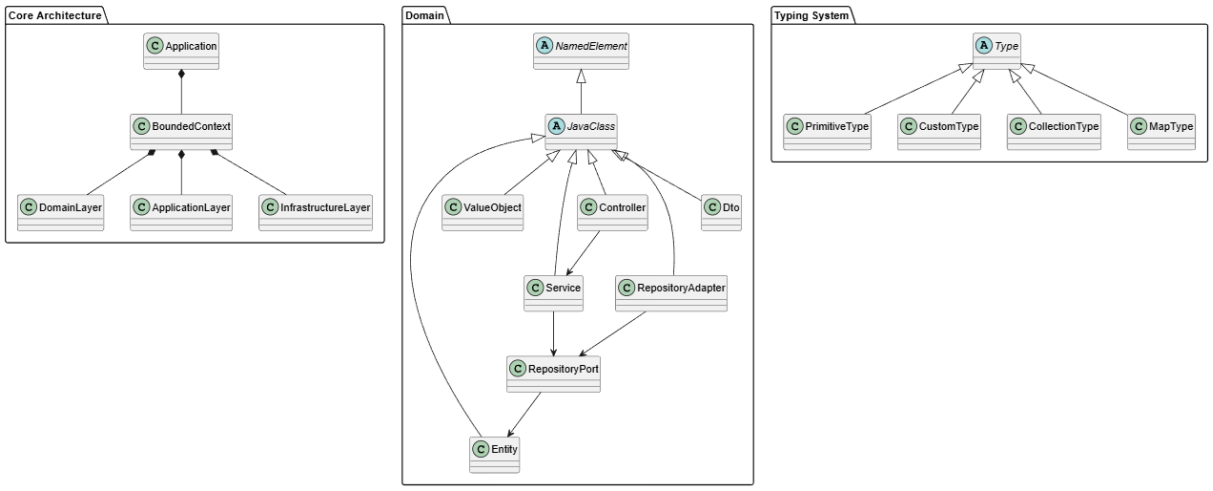

Figure 1: General overview of the HEXAGEN-SMM metamodel.

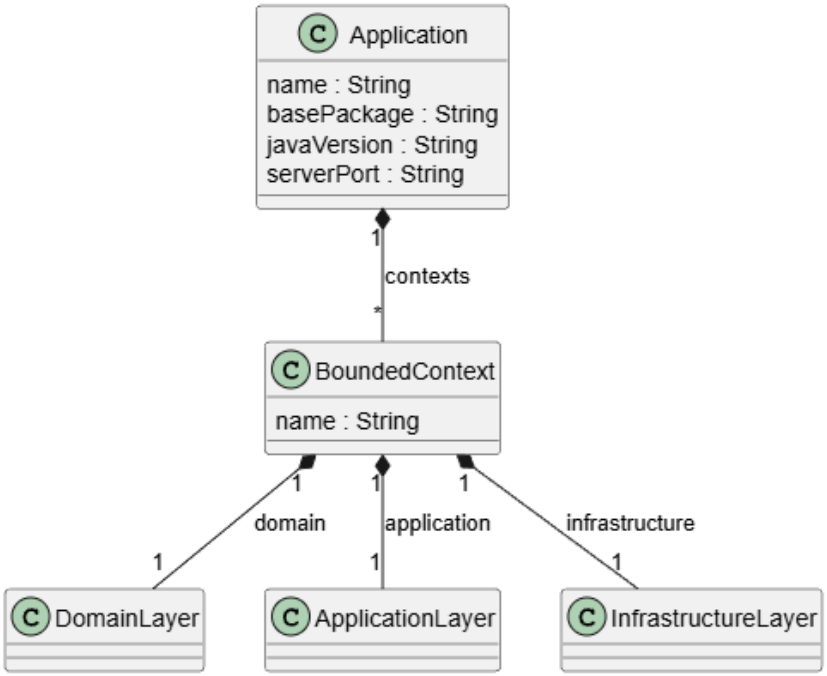

Figure 2: Application containment hierarchy: Application → BoundedContext → Layers.

8.1 Enumerations

8.1.1 Enumeration `HttpMethod`

Represents the standard HTTP verbs used to classify REST endpoints.

Literal	Description
`GET`	Retrieves a resource or collection of resources.
`POST`	Creates a new resource.
`PUT`	Updates an existing resource (full replacement).
`DELETE`	Deletes an existing resource.

8.1.2 Enumeration `ParamKind`

Represents the binding origin of a parameter in a REST endpoint.

Literal	Description
`PATH`	Bound to a path segment (e.g., `/books/{id}`). Generates `@PathVariable`.
`QUERY`	Bound to a query string entry (e.g., `?author=X`). Generates `@RequestParam`.
`BODY`	Bound to the request body. Generates `@RequestBody`.

8.1.3 Enumeration `PrimitiveKind`

Represents the primitive Java data types supported by the type system.

Literal	Java equivalent	Description
`STRING`	`String`	Character sequences.
`INTEGER`	`Integer`	32-bit integer numbers.
`LONG`	`Long`	64-bit integer numbers.
`BOOLEAN`	`Boolean`	Logical true/false values.
`DOUBLE`	`Double`	64-bit floating-point numbers.
`DATE`	`LocalDate`	Calendar dates.
`UUID`	`UUID`	Universally unique identifiers; commonly used as entity identity type.

8.1.4 Enumeration `CollectionKind`

Literal	Java equivalent	Description
`LIST`	`List<T>`	Ordered, allows duplicates.
`SET`	`Set<T>`	Unordered, no duplicates.

8.2 Type System

The type system follows the Composite design pattern around the abstract metaclass `Type`. Every feature that requires a type declaration references a concrete subtype of `Type`.

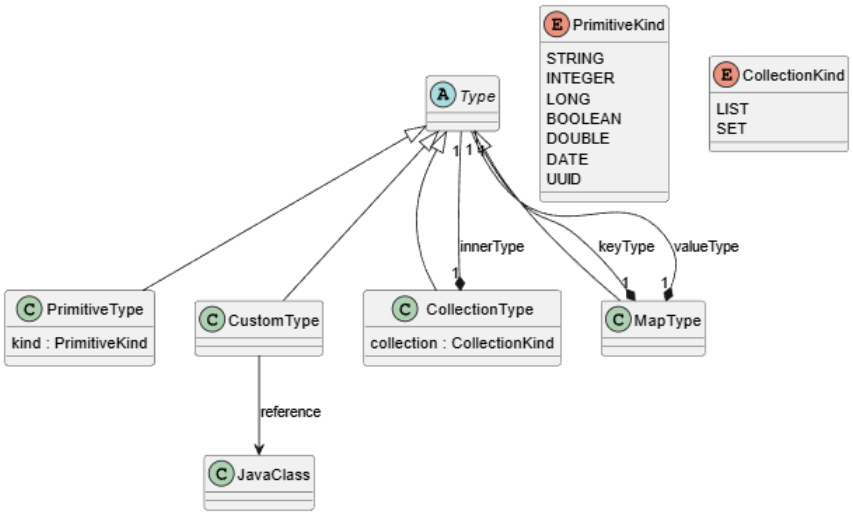

Figure 3: Type system hierarchy.

8.2.1 Abstract Metaclass *Type*

Root of the type hierarchy. Abstract — cannot be instantiated directly. **Superclass:** none. **Attributes:** none. **References:** none.

8.2.2 Metaclass `PrimitiveType`

Represents one of the seven primitive data kinds defined in `PrimitiveKind`. **Superclass:** *Type*.

Name	Type	Multiplicity	Description
kind	PrimitiveKind	0..1	The specific primitive kind.

8.2.3 Metaclass `CustomType`

Represents a type defined by another class in the model. Enables cross-references for method parameters and return types. **Superclass:** *Type*.

References:

Name	Type	Opposite	Aggregation	Description
reference	JavaClass	(none)	Non-containment	The modeled class this type refers to.

8.2.4 Metaclass `CollectionType`

Represents a typed collection (`List` or `Set`) wrapping an inner element type. **Superclass:** *Type*.

Attributes:

Name	Type	Multiplicity	Description
collection	CollectionKind	0..1	Whether the collection is LIST or SET.

References:

Name	Type	Opposite	Aggregation	Description
innerType	Type	(none)	Containment	The type of elements stored in this collection.

8.2.5 Metaclass `MapType`

Represents a `Map<K,V>` type with explicit key and value types. **Superclass:** *Type*.

References:

Name	Type	Opposite	Aggregation	Description
keyType	Type	(none)	Containment	The type of the map keys.
valueType	Type	(none)	Containment	The type of the map values.

8.3 Structural Abstractions

8.3.1 Abstract Metaclass *NamedElement*

Base abstraction for every model element that carries a name. **Superclass:** none.

Name	Type	Multiplicity	Description
name	EString	0..1	The name of the element. MUST be non-empty for all concrete instances.

8.3.2 Abstract Metaclass *JavaClass*

Base abstraction for all metaclasses corresponding to a generated Java class. **Superclass:** *NamedElement*.

References:

Name	Type	Opposite	Aggregation	Description
attributes	Attribute	(none)	Containment	Typed fields declared in this class.
methods	Method	(none)	Containment	Methods declared in this class.

8.4 Hexagonal Architecture

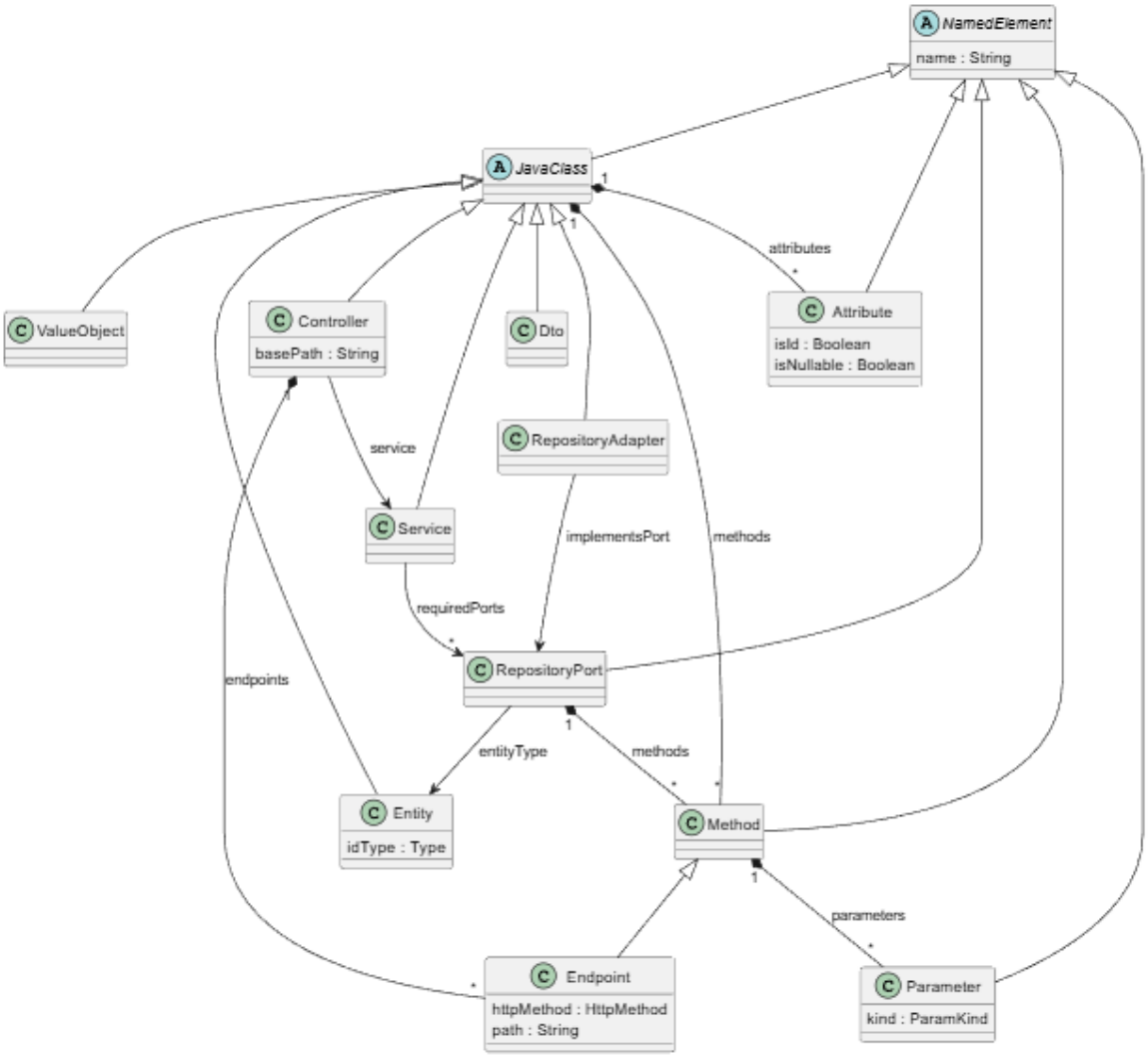

Figure 4: Core metamodel class diagram with attributes and references.

8.4.1 Metaclass `Application`

Root element of the model. Represents the entire Spring Boot application. **Superclass:** none (root metaclass).

Attributes:

Name	Type	Multiplicity	Description
name	EString	0..1	The application name (e.g., `LibraryApp`).
basePackage	EString	0..1	The root Java package (e.g., `com.spring.app`).
javaVersion	EString	0..1	The Java version to target (e.g., `21`).
serverPort	EString	0..1	The embedded server port (e.g., `8080`).

References:

Name	Type	Opposite	Aggregation	Description
contexts	BoundedContext	(none)	Containment	The bounded contexts composing this application.

Constraints: `name` MUST NOT be empty. `basePackage` MUST NOT be empty. MUST contain at least one `BoundedContext`.

8.4.2 Metaclass `BoundedContext`

Represents a cohesive business module organized into exactly one instance of each hexagonal layer. **Superclass:** none.

Attributes: `name : EString [0..1]` — The name of this bounded context.

References:

Name	Type	Opposite	Aggregation	Description
domain	DomainLayer	(none)	Containment	The domain layer.
application	ApplicationLayer	(none)	Containment	The application layer.
infrastructure	InfrastructureLayer	(none)	Containment	The infrastructure layer.

8.4.3 Metaclass `DomainLayer`

Container for entities, value objects, and repository port interfaces. **Superclass:** none.

References:

Name	Type	Opposite	Aggregation	Description
entities	Entity	(none)	Containment	Domain entities.
valueObjects	ValueObject	(none)	Containment	Value objects.
repositoryPorts	RepositoryPort	(none)	Containment	Repository port interfaces.

8.4.4 Metaclass `ApplicationLayer`

Container for the service classes of the bounded context. **Superclass:** none. *References:* `services : Service [*]` (Containment) — The application services.

8.4.5 Metaclass `InfrastructureLayer`

Container for all infrastructure adapters and DTOs. **Superclass:** none.

References:

Name	Type	Opposite	Aggregation	Description
controllers	Controller	(none)	Containment	REST inbound adapters.
repositoryAdapters	RepositoryAdapter	(none)	Containment	Outbound adapters.
dtos	Dto	(none)	Containment	Data Transfer Objects.

8.5 Domain Layer Components

8.5.1 Metaclass `Entity`

Represents a domain object with a unique, persistent identity. **Superclass:** *JavaClass*.

References:

Name	Type	Opposite	Aggregation	Description
idType	Type	(none)	Containment	The type of the entity's primary identifier (commonly `PrimitiveType` with `kind = UUID`).

Constraints: `idType` MUST be present. `name` MUST NOT be empty.

8.5.2 Metaclass `ValueObject`

Domain object with no identity, defined entirely by its attributes. Equality is based on attribute values. **Superclass:** *JavaClass*. No additional attributes or references.

8.5.3 Metaclass `RepositoryPort`

Java interface in the domain layer declaring data access contracts for a specific Entity. Technology-agnostic. **Superclass:** *NamedElement*.

References:

Name	Type	Opposite	Aggregation	Description
entityType	Entity	(none)	Non-containment	The entity this port provides access to.
idType	Type	(none)	Containment	The type of the entity identifier.
methods	Method	(none)	Containment	The data access methods declared in this port.

Constraints: `entityType` MUST be present. `name` MUST NOT be empty.

8.6 Application Layer Components

8.6.1 Metaclass `Service`

Spring Boot service class (`@Service`) implementing one or more use cases. Depends on Repository Ports via constructor injection. **Superclass:** *JavaClass*.

References:

Name	Type	Opposite	Aggregation	Description
requiredPorts	RepositoryPort	(none)	Non-containment	The repository ports this service depends on.

Constraints: `requiredPorts` MUST contain at least one port. `name` MUST NOT be empty.

8.7 Infrastructure Layer Components

8.7.1 Metaclass `Controller`

Spring Boot REST controller (`@RestController`) acting as an inbound adapter. Delegates all processing to a referenced Service. **Superclass:** *JavaClass*.

Attributes:

Name	Type	Multiplicity	Description
basePath	EString	0..1	URL prefix for all endpoints (e.g., `/api/books`).

References:

Name	Type	Opposite	Aggregation	Description
service	Service	(none)	Non-containment	The service this controller delegates to.
endpoints	Endpoint	(none)	Containment	The HTTP endpoints this controller exposes.

Constraints: `basePath` MUST NOT be empty. `service` MUST be present. `name` MUST NOT be empty.

8.7.2 Metaclass `RepositoryAdapter`

Concrete implementation of a Repository Port. Acts as an outbound adapter. **Superclass:** *JavaClass*.

References:

Name	Type	Opposite	Aggregation	Description
implementsPort	RepositoryPort	(none)	Non-containment	The port interface this adapter implements.

Constraints: `implementsPort` MUST be present. `name` MUST NOT be empty.

8.7.3 Metaclass `Dto`

Data Transfer Object used to carry data across layer boundaries. **Superclass:** *JavaClass*. No additional attributes or references.

Constraints: `name` MUST NOT be empty.

8.8 Class Members

8.8.1 Metaclass `Attribute`

Typed field in a Java class. **Superclass:** *NamedElement*.

Attributes:

Name	Type	Multiplicity	Description
isId	Boolean	0..1	True if this is the primary identifier field.
isNullable	Boolean	0..1	True if this field may hold null.

References: `type : Type [1]` (Containment) — The fully modeled type of this attribute.

Constraints: `name` and `type` MUST be present.

8.8.2 Metaclass `Method`

Method declared in a Java class or repository port interface. **Superclass:** *NamedElement*.

References:

Name	Type	Opposite	Aggregation	Description
returnType	Type	(none)	Containment	The return type of this method.
parameters	Parameter	(none)	Containment	The list of parameters.

8.8.3 Metaclass `Endpoint`

Specialization of `Method` representing a specific HTTP operation in a Controller. **Superclass:** *Method*.

Attributes:

Name	Type	Multiplicity	Description
httpMethod	HttpMethod	0..1	The HTTP verb for this operation.
path	EString	0..1	The URL path relative to the controller's `basePath`.

Constraints: `httpMethod` MUST be defined. `path` MUST NOT be empty.

8.8.4 Metaclass `Parameter`

Parameter of a method or endpoint. **Superclass:** *NamedElement*.

Attributes: `kind : ParamKind [0..1]` — The binding kind (PATH, QUERY, BODY).

References: `type : Type [1]` (Containment) — The fully modeled type of this parameter.

Constraints: `name` and `type` MUST be present.

9 Example: Library Management System

The following instance model represents a Library Management System with a single Bounded Context containing a `Book` entity, a `Loan` entity, and a `GET /api/books` endpoint.

9.1 Visual Model

The visual model is provided in the accompanying Sirius `.aird` diagram file. See Figure 1 for the general overview and Figure 4 for the detailed class structure.

9.2 Instance Model in XMI

Listing 1: Library Management System instance model (library-example.xmi).

```
<?xml version="1.0" encoding="UTF-8"?>
<smm:Application xmi:version="2.0"
    xmlns:xmi="http://www.omg.org/XMI"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:smm="smm"
    xsi:schemaLocation="smm smm.ecore"
    name="LibraryApp"
    basePackage="com.spring.app"
    javaVersion="21"
    serverPort="8080">
  <contexts>
    <domain>
      <entities name="Book">
        <attributes name="title">
          <type xsi:type="smm:PrimitiveType"/>
        </attributes>
        <idType xsi:type="smm:PrimitiveType" kind="UUID"/>
      </entities>
      <entities name="Loan">
        <attributes name="book">
          <type xsi:type="smm:CustomType"
                reference="//@contexts.0/@domain/@entities.0"/>
        </attributes>
      </entities>
      <repositoryPorts name="BookRepositoryPort"
                       entityType="//@contexts.0/@domain/@entities.0">
        <methods name="findAll">
```

```
          <returnType xsi:type="smm:CollectionType">
            <innerType xsi:type="smm:CustomType"
                       reference="//@contexts.0/@domain/@entities.0"/>
          </returnType>
        </methods>
      </repositoryPorts>
    </domain>
    <application>
      <services name="LibraryService"
                requiredPorts="//@contexts.0/@domain/@repositoryPorts.0"/>
    </application>
    <infrastructure>
      <controllers name="BookController"
                   basePath="/api/books"
                   service="//@contexts.0/@application/@services.0">
        <endpoints name="getBooks" path="/">
          <returnType xsi:type="smm:CollectionType">
            <innerType xsi:type="smm:CustomType"
                       reference="//@contexts.0/@infrastructure/@dtos.0"/>
          </returnType>
        </endpoints>
      </controllers>
      <repositoryAdapters name="JsonBookRepositoryAdapter"
                          implementsPort="//@contexts.0/@domain/
   @repositoryPorts.0"/>
      <dtos name="BookDto"/>
    </infrastructure>
  </contexts>
</smm:Application>
```

9.3 Mapping to Metaclasses

XMI Element	Metaclass	Explanation
`<smm:Application name="LibraryApp">`	Application	Root element with application metadata.
`<contexts>`	BoundedContext	Single bounded context of the library domain.
`<domain>`	DomainLayer	Domain layer: entities and ports.
`<entities name="Book">`	Entity	Book domain entity with UUID identity.
`<idType kind="UUID"/>`	PrimitiveType	UUID identity type of Book.
`<attributes name="title">`	Attribute	The `title` field of Book.
`<type xsi:type="smm:PrimitiveType"/>`	PrimitiveType	Type of the `title` attribute.
`<entities name="Loan">`	Entity	Loan entity referencing Book.

XMI Element	Metaclass	Explanation
`<type xsi:type="smm:CustomType" reference="..."/>`	CustomType	Cross-reference to Book as type of `book` attribute.
`<repositoryPorts name="BookRepositoryPort">`	RepositoryPort	Interface contract for Book data access.
`<methods name="findAll">`	Method	The `findAll` operation of the port.
`<returnType xsi:type="smm:CollectionType">`	CollectionType	Returns a collection of books.
`<innerType xsi:type="smm:CustomType"/>`	CustomType	Element type of the collection (Book).
`<application>`	ApplicationLayer	Application layer: service classes.
`<services name="LibraryService">`	Service	Use case depending on BookRepositoryPort.
`<infrastructure>`	InfrastructureLayer	Infrastructure layer: adapters and DTOs.
`<controllers name="BookController">`	Controller	REST inbound adapter, delegates to LibraryService.
`<endpoints name="getBooks" path="/">`	Endpoint	HTTP GET at `/api/books/`.
`<repositoryAdapters name="JsonBook...">`	RepositoryAdapter	Outbound adapter implementing BookRepositoryPort.
`<dtos name="BookDto"/>`	Dto	DTO for transferring Book data across layers.

10 Well-Formedness Constraints

The following constraints define conditions that a model instance MUST satisfy to be considered valid. They are formalized in the accompanying EVL file (`hexagen-smm.evl`).

WFC-1: ApplicationHasName — An `Application` MUST have a non-empty `name`. *Rationale:* The name drives generated configuration files and the Spring Boot main class.

WFC-2: ApplicationHasBasePackage — An `Application` MUST have a non-empty `basePackage`. *Rationale:* The base package is the root namespace for all generated Java classes.

WFC-3: ApplicationHasContexts — An `Application` MUST contain at least one `BoundedContext`. *Rationale:* An application with no bounded context has no domain model.

WFC-4: BoundedContextHasName — A `BoundedContext` MUST have a non-empty `name`. *Rationale:* Used as a Java sub-package and logical grouping identifier.

WFC-5: EntityHasIdType — An `Entity` MUST have a defined `idType`. *Rationale:*

Every entity requires a persistent identity; without it, no retrieval logic can be generated.

WFC-6: EntityHasName — An `Entity` MUST have a non-empty `name`. *Rationale:* Drives the generated Java class name.

WFC-7: RepositoryPortHasEntityType — A `RepositoryPort` MUST reference an `Entity` via `entityType`. *Rationale:* A port without an entity is semantically meaningless.

WFC-8: RepositoryPortHasName — A `RepositoryPort` MUST have a non-empty `name`. *Rationale:* Used to generate the Java interface name.

WFC-9: ServiceHasRequiredPorts — A `Service` MUST reference at least one `RepositoryPort`. *Rationale:* A service with no port dependencies is structurally empty.

WFC-10: ServiceHasName — A `Service` MUST have a non-empty `name`. *Rationale:* Generates the Java class name and the `@Service` target.

WFC-11: ControllerHasService — A `Controller` MUST reference a `Service`. *Rationale:* A controller with no service cannot process requests.

WFC-12: ControllerHasBasePath — A `Controller` MUST have a non-empty `basePath`. *Rationale:* Required to generate `@RequestMapping`.

WFC-13: ControllerHasName — A `Controller` MUST have a non-empty `name`. *Rationale:* Generates the Java class name.

WFC-14: RepositoryAdapterImplementsPort — A `RepositoryAdapter` MUST reference a `RepositoryPort`. *Rationale:* An adapter with no port breaks the Ports and Adapters contract.

WFC-15: RepositoryAdapterHasName — A `RepositoryAdapter` MUST have a non-empty `name`. *Rationale:* Generates the Java class name.

WFC-16: EndpointHasPath — An `Endpoint` MUST have a non-empty `path`. *Rationale:* Without a path, no HTTP mapping annotation can be generated.

WFC-17: EndpointHasHttpMethod — An `Endpoint` MUST have a defined `httpMethod`. *Rationale:* The HTTP method determines the annotation to generate (`@GetMapping`, etc.).

WFC-18: EndpointHasName — An `Endpoint` MUST have a non-empty `name`. *Rationale:* Generates the Java method name in the controller class.

WFC-19: AttributeHasName — An `Attribute` MUST have a non-empty `name`. *Rationale:* Generates the Java field name.

WFC-20: AttributeHasType — An `Attribute` MUST have a defined `type`. *Rationale:* A field without a declared type cannot be valid Java.

WFC-21: ParameterHasName — A `Parameter` MUST have a non-empty `name`. *Rationale:* Used as the Java parameter name in method signatures.

WFC-22: ParameterHasType — A `Parameter` MUST have a defined `type`. *Rationale:* Required for a valid Java method signature.

WFC-23: DtoHasName — A `Dto` MUST have a non-empty `name`. *Rationale:* Generates the Java class name for the DTO.

WFC-24: CollectionTypeHasInnerType — A `CollectionType` MUST have a defined `innerType`. *Rationale:* Raw (unparameterized) collections are invalid in this metamodel.

WFC-25: MapTypeHasKeyAndValueTypes — A `MapType` MUST have both `keyType` and `valueType` defined. *Rationale:* A map without explicit key/value types cannot be rendered as a valid generic Java type.

References

[1] Cockburn, A. (2005). *Hexagonal Architecture*. Retrieved from https://alistair.cockburn.us/hexagonal-architecture/

[2] Pivotal Software. (2024). *Spring Boot Reference Documentation*. https://spring.io/projects/spring-boot

[3] Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.

[4] Kolovos, D., Rose, L., García-Domínguez, A., Paige, R. (2024). *The Epsilon Book*. Chapters 4 (EVL) and 7 (EGL). https://eclipse.dev/epsilon/doc/book/

[5] OMG. (2019). *MOF 2.5.1 — Meta-Object Facility Core Specification*. formal/19-10-01.

[6] Steinberg, D., Budinsky, F., Paternostro, M., Merks, E. (2008). *EMF: Eclipse Modeling Framework*. Addison-Wesley.